

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

SYN Flood Pattern Investigation Using TShark

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 10/09/2026

EXECUTIVE SUMMARY

This assignment investigated TCP SYN connection activity using TShark within an authorized Kali Linux laboratory environment. A local Apache web server running on 127.0.0.1:80 was used as the target, ensuring that all traffic generated during the exercise remained within the controlled laboratory environment.

A normal HTTP connection was first captured to establish a baseline for expected TCP behaviour. The baseline demonstrated the normal TCP three-way handshake, HTTP communication, and connection termination through FIN/ACK exchanges. A separate bounded SYN simulation was then conducted using Scapy and restricted to four SYN packets, in accordance with the laboratory safety requirements.

TShark was used to identify the initial SYN packets, SYN-ACK responses, ACK/RST behaviour, source-port distribution, packet counts, and packet timing. The analysis identified four SYN packets from 127.0.0.1 to 127.0.0.1:80, four corresponding SYN-ACK responses, and four subsequent RST packets. The four SYN packets originated from four unique source ports and occurred within an approximately 1.886-millisecond interval.

The findings demonstrate the TCP characteristics of the controlled SYN activity but do not establish a real SYN-flood or denial-of-service condition. The traffic was intentionally limited to four packets, originated from the local host, and provided no evidence of sustained attack traffic, distributed sources, or service degradation. The activity was therefore classified as a bounded SYN-pattern simulation conducted within an authorized laboratory environment.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to develop practical digital-forensics and network-analysis skills through the investigation of TCP SYN activity using TShark. The exercise focuses on understanding how TCP connection establishment appears in packet captures and how network traffic can be examined to identify patterns that may be associated with SYN-flood activity.

The assignment also provides practical experience in establishing a normal network-traffic baseline, collecting packet-capture evidence, applying TShark display filters, extracting relevant packet fields, performing quantitative analysis, preserving evidence integrity, and making evidence-based forensic conclusions.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the assignment are to:

1. Establish a normal TCP traffic baseline for an authorized local web service.
2. Capture and examine controlled TCP SYN activity using TShark.
3. Identify initial SYN packets and corresponding SYN-ACK responses using appropriate TCP flag filters.
4. Examine ACK and RST behaviour associated with the controlled connection attempts.
5. Quantify SYN activity by analysing packet counts, source ports, destinations, and packet timing.
6. Compare the controlled SYN activity against the normal TCP baseline.
7. Determine whether the observed traffic provides sufficient evidence to classify the activity as a SYN flood, port scan, or bounded SYN simulation.
8. Apply basic digital-forensic evidence-preservation practices, including organised evidence storage and SHA-256 hashing.
9. Document the investigation using reproducible commands, screenshots, extracted results, and evidence-based findings.
10. Identify appropriate detection and mitigation measures for genuine SYN-flood activity.

METHODOLOGY OF THE ASSIGNMENT

The investigation was conducted in a controlled Kali Linux virtual-machine environment using the local Apache web server as the authorized target. The loopback address 127.0.0.1 and TCP port 80 were used to ensure that the exercise did not involve public or third-party systems.

The investigation began by creating a structured case directory containing separate folders for evidence, working files, reports, scripts, exported results, and screenshots. The required tools were then verified, including TShark, Scapy, Apache, and the underlying Linux environment.

The Apache service was started and verified as listening on TCP port 80. A normal HTTP request was subsequently captured on the loopback interface to establish a baseline of expected TCP behaviour. The baseline was examined to identify the normal three-way handshake, application-layer traffic, and connection termination sequence.

Following the baseline capture, a bounded SYN simulation was performed against the same local Apache service. The simulation was deliberately restricted to four SYN packets. The resulting packet capture was analysed using TShark display filters to identify initial SYN packets, SYN-ACK responses, and subsequent ACK/RST activity.

Quantitative analysis was then performed to determine the number of SYN packets, destination details, unique source ports, and timing of the SYN requests. The observed traffic was compared with the normal baseline and interpreted in the context of SYN-flood characteristics.

Finally, the captured evidence and extracted analysis files were preserved and hashed where applicable. The findings were documented together with appropriate detection and mitigation recommendations. The investigation was conducted without targeting external systems or generating sustained attack traffic.

TOOLS AND RESOURCES USED

The following tools and resources were used during the investigation:

Tool / Resource	Purpose
Kali Linux VM	Controlled forensic and network-analysis environment
TShark 4.6.6	Packet capture, filtering, extraction, and quantitative network analysis

Tool / Resource	Purpose
Wireshark	Packet-analysis environment supporting review of captured network traffic
Scapy 2.7.0	Generation of the four authorized SYN packets for the bounded simulation
Apache HTTP Server 2.4.68	Local authorized TCP/HTTP target for the investigation
Linux ss utility	Verification that Apache was listening on TCP port 80
cURL	Generation and verification of normal HTTP traffic during baseline establishment
SHA-256 (sha256sum)	Integrity verification of captured evidence
Loopback interface (lo)	Isolated network interface used for the authorized local traffic
Terminal / Bash	Execution of investigation and evidence-extraction commands

The primary evidence sources were the normal HTTP baseline packet capture, the bounded SYN-activity packet capture, TShark-generated analysis files, case notes, command outputs, and SHA-256 integrity records.

Laboratory Environment Preparation

The laboratory environment was prepared within an authorized Kali Linux virtual machine for the investigation of TCP SYN activity using TShark. The investigation was designed to examine normal TCP communication and subsequently compare it with a controlled, bounded SYN-packet simulation against a locally hosted Apache web service.

A dedicated directory structure was created to maintain separation between original evidence, working files, exported analysis results, reports, screenshots, and scripts. This structure supports basic forensic evidence-handling practices and makes the investigation outputs easier to trace and review.

The following directories were created under the main laboratory directory:

- Evidence: used for original PCAP files and preserved evidence.
- Working: used for files generated or used during analysis.
- Exported: used for extracted or exported data.

- Reports: used for TShark analysis results, notes, hashes, and other report materials.
- Screenshots: used to preserve screenshots serving as visual evidence of the investigation steps.
- Scripts: used for the controlled Scapy script required for the bounded SYN-packet simulation.

The screenshot shows a terminal window in a Kali Linux virtual machine. The user is in the directory ~/SBT-DF203-Lab3. They execute a command to create a directory structure: mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}. Then they run cd ~/SBT-DF203-Lab3, pwd (showing /home/kali/SBT-DF203-Lab3), and ls (showing evidence, exported, reports, screenshots, scripts, working). Finally, they run find . -maxdepth 1 -type d -print, which lists the created directories: ./screenshots, ./scripts, ./working, ./reports, ./evidence, and ./exported.

```

kali@kali: ~/SBT-DF203-Lab3
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
kali@kali: ~/SBT-DF203-Lab3
$ cd ~/SBT-DF203-Lab3
kali@kali: ~/SBT-DF203-Lab3
$ pwd
/home/kali/SBT-DF203-Lab3
kali@kali: ~/SBT-DF203-Lab3
$ ls
evidence  exported  reports  screenshots  scripts  working
kali@kali: ~/SBT-DF203-Lab3
$ find . -maxdepth 1 -type d -print
.
./screenshots
./scripts
./working
./reports
./evidence
./exported
kali@kali: ~/SBT-DF203-Lab3
$

```

Tool and Environment Verification

Following the preparation of the laboratory directory structure, the required tools for the investigation were installed and verified within the Kali Linux virtual machine.

TShark, Scapy, and Apache were selected because they provide the required capabilities for packet capture and analysis, controlled TCP traffic generation, and establishment of a local web service for the investigation.

The installed components were verified as follows:

Component	Verified Version / Result
TShark/Wireshark	4.6.6
Scapy	2.7.0
Apache HTTP Server	2.4.68
Linux Kernel	6.12.33+kali-amd64

Component	Verified Version / Result
Scapy Python Import	Scapy OK

TShark successfully returned its version and build information, confirming that the packet-analysis utility was available. Scapy was also successfully imported using Python, returning Scapy OK. Apache was confirmed to be installed with version 2.4.68.

The environment verification established that the required software components were available before proceeding to the network-service and packet-capture stages of the investigation.

```

kali@kali:~/SBT-DF203-Lab3$ tshark --version
Warning: program compiled against libxml 215 using older 214
TShark (Wireshark) 4.6.6.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even For MERCHANTABILITY or FITNESS For A PARTICULAR PURPOSE.

Compile-time info:
  Bit widths: 64-Bit
  Compiler: GCC 15.2.0
  GLib: 2.80.1

With:
  +brotli
  +Gcrypt 1.12.2
  +GnuTLS 3.8.13 and PKCS#11
  +Kerberos (MIT)
  +libnl 3
  +libpcap
  +libsmi 0.4.8
  +libxml2 2.15.2
  +Lua 5.4.8
  +LZ4 1.10.0
  +MaxMind
  +nghttp2 1.69.0
  +nghttp3 1.15.0
  +PCRE2 10.46 2025-08-27
  +POSIX capabilities (Linux)
  +Snappy 1.2.2
  +xxhash 0.8.3
  +zlib 1.3.2
  +Zstandard 1.5.7

Without:
  -zlib-ng

Runtime info:
  OS: Linux 6.12.33+kali-amd64
  CPU: Intel(R) Core(TM) i7-4680U CPU @ 2.10GHz (with SSE4.2)
  Memory: 6894 MB of physical memory
  GLib: 2.84.3
  Locale: LC_TYPE=en_US.UTF-8
  Plugins: supported, 0 loaded

With:
  +brotli 1.1.0
  +c-ares 1.34.5
  +Gcrypt 1.12.2
  +GnuTLS 3.8.9
  +libpcap 1.10.5 (with TPACKET_V3)
  +libsmi 0.4.8
  +LZ4 1.10.0
  +nghttp2 1.64.0
  +nghttp3 1.17.0
  +PCRE2 10.45 2025-02-05
  +xxhash 803
  +zlib 1.3.1
  +Zstandard 1.5.7

```

```
kali-linux-2025.2-vmware... X
File Actions Edit View Help
Compiler: GCC 15.2.0
GLib: 2.88.1
With:
+brotili                               +MaxMind
+Gcrypt 1.12.2                         +nghttp2 1.69.0
+GnuTLS 3.8.13 and PKCS#11             +nghttp3 1.15.0
+Kerberos (MIT)                       +PCRE2 10.46 2025-08-27
+libnl 3                               +POSIX capabilities (Linux)
+libpcap                               +Snappy 1.2.2
+libsmi 0.4.8                         +xxhash 0.8.3
+libxml2 2.15.2                       +zlib 1.3.2
+Lua 5.4.8                             +Zstandard 1.5.7
+LZ4 1.10.0
Without:
-zlib-ng

Runtime info:
OS: Linux 6.12.33+kali-amd64
CPU: Intel(R) Core(TM) i7-4600U CPU @ 2.10GHz (with SSE4.2)
Memory: 6894 MB of physical memory
GLib: 2.84.3
Locale: LC_TYPE=en_US.UTF-8
Plugins: supported, 0 loaded
With:
+brotili 1.1.0                         +nghttp2 1.64.0
+c-ares 1.34.5                       +nghttp3 1.17.0
+Gcrypt 1.12.2                       +PCRE2 10.45 2025-02-05
+GnuTLS 3.8.9                       +xxhash 803
+libpcap 1.10.5 (with TPACKET_V3)     +zlib 1.3.1
+libsmi 0.4.8                       +Zstandard 1.5.7
+LZ4 1.10.0

(kali@kali)-[~/SBT-DF203-Lab3]
$ python3 -c "import scapy; print('Scapy OK')"
Scapy OK

(kali@kali)-[~/SBT-DF203-Lab3]
$ apache2 -v
Server version: Apache/2.4.68 (Debian)
Server built: 2026-06-11T18:22:17

(kali@kali)-[~/SBT-DF203-Lab3]
$
```

TShark successfully returned its version and build information, confirming that the packet-analysis utility was available. Scapy was also successfully imported using Python, returning Scapy OK. Apache was confirmed to be installed with version 2.4.68.

The environment verification established that the required software components were available before proceeding to the network-service and packet-capture stages of the investigation.

Local Apache Service Verification

Before collecting network evidence, the local Apache web service was verified as the authorized target for the investigation. The service was configured to listen on TCP port 80 and was tested using the loopback address 127.0.0.1.

```
(kali@kali) [~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali@kali) [~/SBT-DF203-Lab3]
$ sudo ss -ltnp | grep ':80'
LISTEN 0      511          *:*        users:((("apache2",pid=43851,fd=4),("apache2",pid=43850,fd=4),("apache2",pid=43849,fd=4),("apache2",pid=43848,fd=4),("apache2",pid=43847,fd=4),("apache2",pid=1390,fd=4)))

(kali@kali) [~/SBT-DF203-Lab3]
$ curl --no-keepalive http://127.0.0.1/
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Apache2 Debian Default Page: It works</title>
<style type="text/css" media="screen">
* {
```

This confirms that the Apache process is actively listening on TCP port 80. The *:80 binding indicates that Apache is listening on port 80 on the available local interfaces, while the investigation itself will use the explicitly authorized loopback target 127.0.0.1.

The local service was then tested with curl --no-keepalive <http://127.0.0.1/>. The server returned the Apache2 Debian default page containing Apache2 Debian Default Page.

```
(kali@kali) [~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali@kali) [~/SBT-DF203-Lab3]
$ sudo ss -ltnp | grep ':80'
LISTEN 0      511          *:*        users:((("apache2",pid=43851,fd=4),("apache2",pid=43850,fd=4),("apache2",pid=43849,fd=4),("apache2",pid=43848,fd=4),("apache2",pid=43847,fd=4),("apache2",pid=1390,fd=4)))

(kali@kali) [~/SBT-DF203-Lab3]
$ curl --no-keepalive http://127.0.0.1/
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Apache2 Debian Default Page: It works</title>
<style type="text/css" media="screen">
* {
```

This confirms that the HTTP service was reachable and functioning before the packet-capture phase.

Case Identification and Evidence Worksheet

Before collecting network traffic, the investigation environment and case information were documented to establish the identity, scope, and context of the evidence.

The following information was also recorded:

Field	Recorded Information
Case/Lab Identifier	SBT-DF203-Lab3
Student	Kafayat Animashawun
Course	SBT-DF203 Basic Networking Skills for Digital Forensics
Lab	Lab 3 - SYN Flood Pattern Investigation Using TShark
Environment	Kali Linux VM
Target	Local Apache service
Target IP	127.0.0.1
Target Port	TCP/80
Scope	Authorized isolated laboratory environment

```

File Actions Edit View Help
GNU nano 9.1 reports/case_notes.txt *
Case/Lab Identifier: SBT-DF203-Lab3
Student: Kafayat Omolara Animashawun
Course: SBT-DF203 Basic Networking Skills for Digital Forensics
Lab: Lab 3 - SYN Flood Pattern Investigation Using TShark
Environment: Kali Linux VM
Target: Local Apache service
Target IP: 127.0.0.1
Target Port: TCP/80
Scope: Authorized isolated laboratory environment

Write to File: reports/case_notes.txt
^G Help      M-D DOS Format  M-A Append     ^O Discard buffer
^C Cancel    M-B Backup File M-P Prepend    ^T Browse

```

The case notes provide a consistent identifier for the investigation and clearly define the authorized target and scope. In particular, specifying 127.0.0.1:80 establishes that the subsequent traffic-generation and packet-analysis activities are restricted to the local Apache service within the isolated laboratory environment.

The system date, hostname, and network interface information were also recorded to establish the environment and timestamp associated with the investigation. The network information was collected for identification purposes and does not need to be reproduced in full in the final report.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ date
Fri Sep 11 12:24:16 PM EDT 2026
$ hostname
kali
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:77:75:3c brd ff:ff:ff:ff:ff:ff
    inet 192.168.232.128/24 brd 192.168.232.255 scope global dynamic noprefixroute eth0
        valid_lft 943sec preferred_lft 943sec
    inet6 fe80::d5cc:fb25:aa2d:b901/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:ad:86:0d:b4 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
4: br-d2696585dc9a: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:37:79:ad:f5:61 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-d2696585dc9a
        valid_lft forever preferred_lft forever
    inet6 fe80::37:79ff:fead:f861/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
5: br-ec90d8c3abfb: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 0e:91:8c:64:bd:f3 brd ff:ff:ff:ff:ff:ff
    inet 172.19.0.1/16 brd 172.19.255.255 scope global br-ec90d8c3abfb
        valid_lft forever preferred_lft forever
6: vethe5407a9@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-d2696585dc9a state UP group default
    link/ether d2:15:ce:3d:81:ce brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::d015:ceff:fe3d:81ce/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Normal TCP/HTTP Baseline Capture

Before conducting the controlled SYN traffic simulation, a normal TCP/HTTP baseline was established against the authorized local Apache service at 127.0.0.1:80. The purpose of this stage was to capture legitimate traffic to the target service and provide a reference for comparison with the subsequent controlled SYN activity.

TShark was initially configured to capture traffic on the loopback interface using the capture filter tcp port 80 for a duration of 15 seconds. The initial attempt was made to save the capture directly into the laboratory's evidence directory:

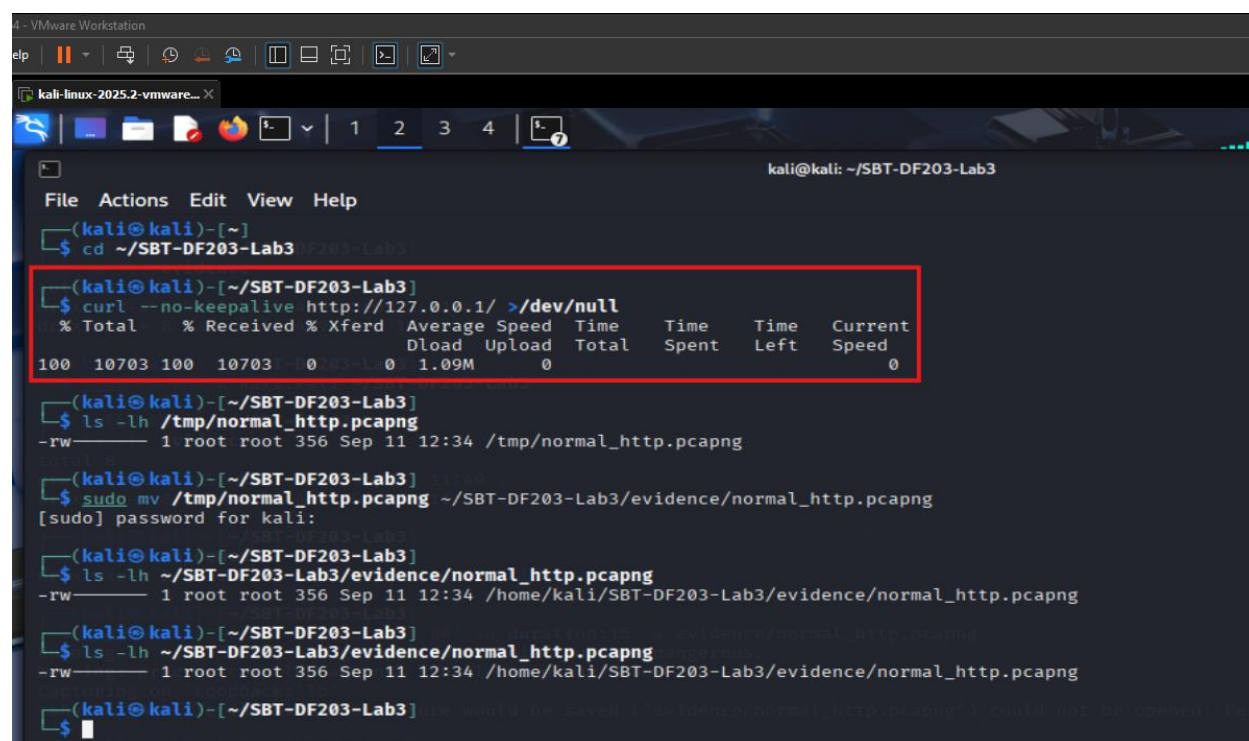
```
(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
tshark: The file to which the capture would be saved ("evidence/normal_http.pcapng") could not be opened: Permission denied.

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /tmp/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /tmp/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
```

The capture interface was successfully initialized; however, TShark returned a Permission denied error when attempting to create the output file in the evidence directory. The error prevented the capture from being written to the intended location.

To avoid changing the evidence-directory permissions unnecessarily, the capture destination was changed to the system temporary directory. TShark was then started with the same loopback interface, TCP/80 filter, and 15-second capture duration: `sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /tmp/normal_http.pcapng`. TShark successfully began capturing traffic on the loopback interface. During the capture window, a normal HTTP request was generated against the authorized local Apache service using:



```
kali@kali: ~/SBT-DF203-Lab3
File Actions Edit View Help
(kali@kali)-[~]
$ cd ~/SBT-DF203-Lab3
(kali@kali)-[~/SBT-DF203-Lab3]
$ curl --no-keepalive http://127.0.0.1/ >/dev/null
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100 10703 100 10703    0  1.09M    0    0    0    0    0    0
(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh /tmp/normal_http.pcapng
-rw----- 1 root root 356 Sep 11 12:34 /tmp/normal_http.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo mv /tmp/normal_http.pcapng ~/SBT-DF203-Lab3/evidence/normal_http.pcapng
[sudo] password for kali:
(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh ~/SBT-DF203-Lab3/evidence/normal_http.pcapng
-rw----- 1 root root 356 Sep 11 12:34 /home/kali/SBT-DF203-Lab3/evidence/normal_http.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh ~/SBT-DF203-Lab3/evidence/normal_http.pcapng
-rw----- 1 root root 356 Sep 11 12:34 /home/kali/SBT-DF203-Lab3/evidence/normal_http.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

The request completed successfully and transferred 10,703 bytes from the local web server. This confirmed that normal application-layer traffic was generated while the packet capture was active.

After the capture completed, the resulting PCAP file was checked in the temporary directory. The file was present with a size of 356 bytes and was initially owned by the root account because TShark had been executed with sudo:

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh /tmp/normal_http.pcapng
-rw-r--r-- 1 root root 356 Sep 11 12:34 /tmp/normal_http.pcapng

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo mv /tmp/normal_http.pcapng ~/SBT-DF203-Lab3/evidence/normal_http.pcapng
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh ~/SBT-DF203-Lab3/evidence/normal_http.pcapng
-rw-r--r-- 1 root root 356 Sep 11 12:34 /home/kali/SBT-DF203-Lab3/evidence/normal_http.pcapng

(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh ~/SBT-DF203-Lab3/evidence/normal_http.pcapng
-rw-r--r-- 1 root root 356 Sep 11 12:34 /home/kali/SBT-DF203-Lab3/evidence/normal_http.pcapng

(kali@kali)-[~/SBT-DF203-Lab3]
$
```

The captured evidence was subsequently moved from the temporary directory into the laboratory evidence directory

Before conducting the controlled SYN activity, a normal HTTP transaction was generated against the authorized local Apache service at 127.0.0.1:80. TShark was used to monitor the loopback interface (lo) with a capture filter restricted to TCP port 80. This baseline was collected to establish the expected TCP connection and HTTP communication pattern against which the subsequent controlled SYN traffic could be compared.

```
VMware Workstation
kali-linux-2025.2-vmware...
kali@kali: ~/SBT-DF203-Lab3

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo ss -ltnp | grep ':80'
LISTEN 0      4096      0.0.0.0:*        users:((("docker-proxy",pid=29434,fd=8))
LISTEN 0      4096      [::]:800        [::]:* users:((("docker-proxy",pid=29440,fd=8))
LISTEN 0      511      *:*            *:* users:((("apache2",pid=1468,fd=4),("apache2",pid=1467,fd=4),("apache2",pid=1466,fd=4),("apache2",pid=1465,fd=4),("apache2",pid=1464,fd=4),("apache2",pid=1322,fd=4))

(kali@kali)-[~/SBT-DF203-Lab3]
$ curl --no-keepalive http://127.0.0.1/>/dev/null 80 echo "Apache HTTP test: SUCCESS"
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed
100 10703 100 10703    0  1.16M    0    0    0    0    0     0
Apache HTTP test: SUCCESS

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80'
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on "Loopback: lo"
1 0.000000000 127.0.0.1 → 127.0.0.1    TCP 74 49908 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1510854239 TSecr=0 WS=128
2 0.000038458 127.0.0.1 → 127.0.0.1    TCP 74 80 → 49908 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1510854239 TSecr=15108542
39 WS=128
3 0.000059821 127.0.0.1 → 127.0.0.1    TCP 66 49908 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1510854239 TSecr=1510854239
4 0.001088975 127.0.0.1 → 127.0.0.1    HTTP 139 GET / HTTP/1.1
5 0.001159123 127.0.0.1 → 127.0.0.1    TCP 66 80 → 49908 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=1510854240 TSecr=1510854240
6 0.005040490 127.0.0.1 → 127.0.0.1    HTTP 11024 HTTP/1.1 200 OK (text/html)
7 0.005129554 127.0.0.1 → 127.0.0.1    TCP 66 49908 → 80 [ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=1510854244 TSecr=1510854244
8 0.006104267 127.0.0.1 → 127.0.0.1    TCP 66 49908 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=1510854245 TSecr=1510854244
9 0.009086500 127.0.0.1 → 127.0.0.1    TCP 66 80 → 49908 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=1510854248 TSecr=1510854245
10 0.009141655 127.0.0.1 → 127.0.0.1    TCP 66 49908 → 80 [ACK] Seq=75 Ack=10960 Win=109440 Len=0 TSval=1510854248 TSecr=1510854248
```

The capture shows a complete TCP connection lifecycle. Frame 1 contains a SYN from the client ephemeral port 49908 to TCP/80, followed by a SYN-ACK from the Apache

service in frame 2. Frame 3 contains the client's ACK, completing the TCP three-way handshake.

This traffic represents normal, successful client-server communication. The presence of the SYN-ACK and subsequent ACK demonstrates that the connection progressed beyond the initial SYN stage and was accepted by the local service. This baseline provides a reference for distinguishing normal completed connections from the later bounded SYN activity.

See below the complete normal HTTP transaction:

Frame	Activity	Meaning
1	[SYN]	Client 49908 requests a TCP connection to port 80
2	[SYN, ACK]	Apache acknowledges and accepts the connection
3	[ACK]	Client completes the TCP three-way handshake
4	GET / HTTP/1.1	Client sends the HTTP request
5	[ACK]	Server acknowledges the request
6	HTTP/1.1 200 OK	Apache returns the web page
7	[ACK]	Client acknowledges the response
8	[FIN, ACK]	Client begins graceful connection termination
9	[FIN, ACK]	Server closes its side
10	[ACK]	Client acknowledges the server's FIN

A normal HTTP connection to the authorized Apache service at 127.0.0.1:80 was captured using TShark on the loopback interface. The capture showed a successful TCP three-way handshake, followed by an HTTP GET request and an HTTP/1.1 200 OK response, confirming normal application-layer communication. The connection then terminated through a FIN-ACK exchange followed by a final ACK, indicating graceful TCP session closure. The baseline PCAP was preserved as evidence/normal_http.pcapng, and its SHA-256 hash was calculated to provide an integrity reference for the evidence.

```
kali@kali: ~/SBT-DF203-Lab3
File Actions Edit View Help

(kali@kali)~[~/SBT-DF203-Lab3]
$ cd ~/SBT-DF203-Lab3
(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -w /tmp/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
10 ^C

(kali@kali)~[~/SBT-DF203-Lab3]
$ ls -lh /tmp/normal_http.pcapng
-rw-r--r-- 1 root root 24K Sep 11 15:52 /tmp/normal_http.pcapng

(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo tshark -r /tmp/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 57920 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1512497532 TSecr=0 WS=128
2 0.000026911 127.0.0.1 → 127.0.0.1 TCP 74 80 → 57920 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1512497532 TSecr=15124975
32 WS=128
3 0.000046371 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1512497532 TSecr=1512497532
4 0.001826029 127.0.0.1 → 127.0.0.1 HTTP 139 GET / HTTP/1.1
5 0.001902973 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=1512497534 TSecr=1512497534
6 0.003577509 127.0.0.1 → 127.0.0.1 HTTP 11024 HTTP/1.1 200 OK (text/html)
7 0.003603616 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=1512497536 TSecr=1512497536
8 0.006060843 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=1512497538 TSecr=1512497538
9 0.006223804 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=1512497538 TSecr=1512497538
10 0.006247130 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=75 Ack=10960 Win=109440 Len=0 TSval=1512497538 TSecr=1512497538
11 0.000027259 127.0.0.1 → 127.0.0.1 TCP 74 [TCP Port numbers reused] 80 → 57920 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval
=1512497532 TSecr=1512497532 WS=128
12 0.000046728 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0 TSval=1512497532 TSecr=1512497532
13 0.001826411 127.0.0.1 → 127.0.0.1 HTTP 139 GET / HTTP/1.1
14 0.001904681 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=1512497534 TSecr=1512497534
15 0.003579237 127.0.0.1 → 127.0.0.1 HTTP 11024 HTTP/1.1 200 OK (text/html)
16 0.003603973 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=74 Ack=10959 Win=855 Len=0 TSval=1512497536 TSecr=1512497536
17 0.006061237 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=855 Len=0 TSval=1512497538 TSecr=1512497538
18 0.006224073 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=1512497538 TSecr=1512497538
19 0.006247382 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=75 Ack=10960 Win=855 Len=0 TSval=1512497538 TSecr=1512497538
```

The baseline PCAP was preserved as evidence/normal_http.pcapng and assigned the SHA-256 hash b64a4e26331fab5548158e46f5ba15554ac7016d340a9fdd5a3b6f6db84be6a3 to support evidence integrity and verification.

```
(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo tshark -r /tmp/normal_http.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 57920 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1512497532 TSecr=0 WS=128
2 0.000026911 127.0.0.1 → 127.0.0.1 TCP 74 80 → 57920 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1512497532 TSecr=15124975
32 WS=128
3 0.000046371 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1512497532 TSecr=1512497532
4 0.001826029 127.0.0.1 → 127.0.0.1 HTTP 139 GET / HTTP/1.1
5 0.001902973 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=1512497534 TSecr=1512497534
6 0.003577509 127.0.0.1 → 127.0.0.1 HTTP 11024 HTTP/1.1 200 OK (text/html)
7 0.003603616 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=1512497536 TSecr=1512497536
8 0.006060843 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=1512497538 TSecr=1512497538
9 0.006223804 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=1512497538 TSecr=1512497538
10 0.006247130 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=75 Ack=10960 Win=109440 Len=0 TSval=1512497538 TSecr=1512497538
11 0.000027259 127.0.0.1 → 127.0.0.1 TCP 74 [TCP Port numbers reused] 80 → 57920 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval
=1512497532 TSecr=1512497532 WS=128
12 0.000046728 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0 TSval=1512497532 TSecr=1512497532
13 0.001826411 127.0.0.1 → 127.0.0.1 HTTP 139 GET / HTTP/1.1
14 0.001904681 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=1512497534 TSecr=1512497534
15 0.003579237 127.0.0.1 → 127.0.0.1 HTTP 11024 HTTP/1.1 200 OK (text/html)
16 0.003603973 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=74 Ack=10959 Win=855 Len=0 TSval=1512497536 TSecr=1512497536
17 0.006061237 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=855 Len=0 TSval=1512497538 TSecr=1512497538
18 0.006224073 127.0.0.1 → 127.0.0.1 TCP 66 80 → 57920 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=1512497538 TSecr=1512497538
19 0.006247382 127.0.0.1 → 127.0.0.1 TCP 66 57920 → 80 [ACK] Seq=75 Ack=10960 Win=855 Len=0 TSval=1512497538 TSecr=1512497538

(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo mv /tmp/normal_http.pcapng evidence/normal_http.pcapng
$ sudo chown kali:kali evidence/normal_http.pcapng
$ sha256sum evidence/normal_http.pcapng | tee reports/normal_http_sha256.txt
b64a4e26331fab5548158e46f5ba15554ac7016d340a9fdd5a3b6f6db84be6a3 evidence/normal_http.pcapng

(kali@kali)~[~/SBT-DF203-Lab3]
```

A controlled SYN activity was conducted against the authorized local Apache service at 127.0.0.1:80 using the Scapy script. The script was configured to send exactly four SYN packets, and the terminal output confirmed that all four authorized training packets were sent successfully. TShark captured the resulting traffic for subsequent analysis.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ nano scripts/bounded_syn.py
(kali@kali)-[~/SBT-DF203-Lab3]
$ cd ~/SBT-DF203-Lab3
$ sudo python3 scripts/bounded_syn.py
Sent 4 authorized training SYN packets to 127.0.0.1:80
(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh /tmp/bounded_syn_activity.pcapng
-rw-r--r-- 1 root root 1.3K Sep 11 16:04 /tmp/bounded_syn_activity.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]

```

The 1.3K PCAP file also confirms that the capture was created successfully.

The captured traffic was filtered for TCP SYN packets without the ACK flag. Four initial SYN packets were identified, all originating from 127.0.0.1 and targeting the Apache service on TCP port 80. Each packet used a different source port, confirming the four SYN packets generated by the controlled simulation.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh evidence/bounded_syn_activity.pcapng
-rw-r--r-- 1 kali kali 1.3K Sep 11 16:04 evidence/bounded_syn_activity.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/bounded_syn_activity.pcapng | tee reports/bounded_syn_sha256.txt
a2104df4278d58d6315bc4422552edaa977351960b300bc9a7b5919d3ce3605e evidence/bounded_syn_activity.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng \
  -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' \
  -T fields \
  -e frame.number -e frame.time_epoch \
  -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv
Warning: program compiled against libxml 215 using older 214
1      1789157062.227471477    127.0.0.1      27322    127.0.0.1      80      0
4      1789157062.228140336    127.0.0.1      4924     127.0.0.1      80      0
7      1789157062.228821336    127.0.0.1      18524    127.0.0.1      80      0
10     1789157062.229357509    127.0.0.1      33781    127.0.0.1      80      0

```

The captured SYN activity was preserved as evidence/bounded_syn_activity.pcapng and assigned a SHA-256 hash of a2104df4278d58d6315bc4422552edaa977351960b300bc9a7b5919d3ce3605e to maintain evidence integrity.

To establish the expected behaviour of a normal TCP connection, traffic between the local host and the Apache web service on 127.0.0.1:80 was captured during a normal HTTP request. TShark was used to filter packets containing the SYN or FIN flags in order to identify the connection establishment and termination stages. The output shows an initial SYN from client port 57920 to TCP port 80, followed by a SYN-ACK from the Apache service. The connection was subsequently terminated through FIN/ACK exchanges. This establishes the normal TCP connection lifecycle against which the subsequent controlled SYN activity was compared.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng \
-Y 'tcp.flags.syn==1 && tcp.flags.ack==1' \
-T fields \
-e frame.number -e frame.time_epoch \
-e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
| tee reports/syn_ack_responses.tsv
Warning: program compiled against libxml 215 using older 214
  2      1789157062.227528516      127.0.0.1      80      127.0.0.1      27322      1
  5      1789157062.228153718      127.0.0.1      80      127.0.0.1      4924      1
  8      1789157062.228834377      127.0.0.1      80      127.0.0.1      18524      1
 11      1789157062.229369929      127.0.0.1      80      127.0.0.1      33781      1

```

The baseline output contains additional entries at frames 11, 17 and 18 representing TShark's subsequent analysis of the captured traffic around TCP port reuse. These entries were not treated as additional independent connection events. The original frames 1–10 were used as the primary baseline sequence because they represent the observed HTTP transaction and its associated TCP lifecycle.

SYN-ACK Response Analysis

Following the baseline capture, the bounded SYN activity was examined using a TShark display filter for packets containing both the SYN and ACK flags. The resulting output identified four SYN-ACK responses from the local Apache service at 127.0.0.1:80. The responses were directed to client source ports 27322, 4924, 18524, and 33781, corresponding to the four SYN packets generated during the authorized simulation. The results therefore confirm that each of the four controlled SYN connection attempts received a SYN-ACK response from the target service.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/normal_http.pcapng \
-Y 'tcp.flags.syn==1 || tcp.flags.fin==1' \
-T fields \
-e frame.number \
-e frame.time \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags
Warning: program compiled against libxml 215 using older 214
  1      2026-09-11T15:50:46.664302176-0400      127.0.0.1      57920      127.0.0.1      80      0x0002
  2      2026-09-11T15:50:46.664329087-0400      127.0.0.1      80      127.0.0.1      57920      0x0012
  8      2026-09-11T15:50:46.670363019-0400      127.0.0.1      57920      127.0.0.1      80      0x0011
  9      2026-09-11T15:50:46.670525980-0400      127.0.0.1      80      127.0.0.1      57920      0x0011
 11      2026-09-11T15:50:46.664329435-0400      127.0.0.1      80      127.0.0.1      57920      0x0012
 17      2026-09-11T15:50:46.670363413-0400      127.0.0.1      57920      127.0.0.1      80      0x0011
 18      2026-09-11T15:50:46.670526249-0400      127.0.0.1      80      127.0.0.1      57920      0x0011

```

The presence of four SYN-ACK responses confirms that the target service was reachable and responded to each simulated connection attempt. However, these observations alone do not establish a SYN flood or denial-of-service condition. The activity was deliberately limited to four packets, and further analysis of ACK/RST

behaviour, packet counts, source-port distribution, and TCP characteristics is required before making a forensic determination.

Following identification of the four SYN-ACK responses, the capture was examined for acknowledgement and reset behaviour. The results showed that each SYN-ACK response was immediately followed by a TCP RST packet from the originating client port. Specifically, frames 2–3, 5–6, 8–9, and 11–12 formed four corresponding SYN-ACK/RST pairs. The RST packets indicate that the simulated connection attempts were terminated before completion of the normal TCP three-way handshake. This differs from the normal baseline, where the SYN-ACK was followed by an ACK and subsequent application-layer communication. The observed pattern is therefore consistent with the deliberately bounded SYN simulation and demonstrates that the four connection attempts did not proceed into normal HTTP sessions.

Each SYN-ACK is immediately followed by a **RST** from the client:

SYN-ACK	SYN-ACK	Following packet	Interpretation
Frame 2	80 → 27322	Frame 3: 27322 → 80, RST	Connection attempt terminated
Frame 5	80 → 4924	Frame 6: 4924 → 80, RST	Connection attempt terminated
Frame 8	80 → 18524	Frame 9: 18524 → 80, RST	Connection attempt terminated
Frame 11	80 → 33781	Frame 12: 33781 → 80, RST	Connection attempt terminated

The important flag values are:

- 0x0012 = SYN + ACK
- 0x0004 = RST

So the pattern for each of the four simulated connection attempts is: SYN → SYN-ACK → RST

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng \
-Y 'tcp.flags.reset=1 || (tcp.flags.ack=1 && tcp.len=0)' \
-T fields \
-e frame.number \
-e frame.time \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags \
| tee reports/ack_reset_candidates.tsv
Warning: program compiled against libxml 215 using older 214
2 2026-09-11T16:04:22.227528516-0400 127.0.0.1 80 127.0.0.1 27322 0x0012
3 2026-09-11T16:04:22.227537673-0400 127.0.0.1 27322 127.0.0.1 80 0x0004
5 2026-09-11T16:04:22.228153718-0400 127.0.0.1 80 127.0.0.1 4924 0x0012
6 2026-09-11T16:04:22.228159176-0400 127.0.0.1 4924 127.0.0.1 80 0x0004
8 2026-09-11T16:04:22.228834377-0400 127.0.0.1 80 127.0.0.1 18524 0x0012
9 2026-09-11T16:04:22.228840355-0400 127.0.0.1 18524 127.0.0.1 80 0x0004
11 2026-09-11T16:04:22.229369929-0400 127.0.0.1 80 127.0.0.1 33781 0x0012
12 2026-09-11T16:04:22.229374933-0400 127.0.0.1 33781 127.0.0.1 80 0x0004

```

Quantitative Analysis of Controlled SYN Activity

The captured traffic was quantified by filtering for initial TCP SYN packets, defined as packets with the SYN flag set and the ACK flag unset. The source and destination IP addresses and destination port were then grouped to determine the volume of SYN attempts directed at the target service.

The analysis identified four SYN packets from 127.0.0.1 to 127.0.0.1:80. The result was therefore 4 SYN packets → 127.0.0.1:80. This corresponds exactly with the four-packet limit defined for the authorized laboratory simulation.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng \
-Y 'tcp.flags.syn=1 && tcp.flags.ack=0' \
-T fields \
-e ip.src -e ip.dst -e tcp.dstport \
| sort | uniq -c | sort -nr \
| tee reports/syn_counts_by_pair.txt
Warning: program compiled against libxml 215 using older 214
4 127.0.0.1 127.0.0.1 80

```

The source-port analysis was then performed to determine whether the SYN packets originated from the same or different ephemeral ports. Four unique source ports were identified: 4924, 18524, 27322, and 33781. The use of four distinct source ports corresponds to the four individual SYN connection attempts generated during the simulation.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng \
-Y 'tcp.flags.syn=1 && tcp.flags.ack=0' \
-T fields \
-e tcp.srcport \
| sort -n | uniq \
| tee reports/unique_syn_source_ports.txt
Warning: program compiled against libxml 215 using older 214
4924
18524
27322
33781
(kali@kali)-[~/SBT-DF203-Lab3]
$

```

The quantitative results establish a small, bounded set of four SYN attempts, all originating from the local host and targeting the authorized Apache service on TCP port 80. Although multiple source ports were used, the traffic volume is far below what would ordinarily be required to characterize a SYN flood. There is also no evidence from this capture alone of sustained high-rate traffic, distributed sources, service exhaustion, or denial of service. Therefore, at this stage, the evidence supports the conclusion that the capture represents a controlled SYN-pattern simulation rather than a demonstrated SYN flood.

SYN Timeline and Packet Rate

To further quantify the controlled SYN activity, the timestamp, source port, source address, destination address, and destination port of each initial SYN packet were extracted from the capture. Four SYN packets were observed at frames 1, 4, 7, and 10, originating from the local host and targeting 127.0.0.1:80. The first SYN was recorded at epoch time 1789157062.227471477, while the fourth was recorded at 1789157062.229357509, giving an overall observed span of approximately 1.886 milliseconds. The four packets therefore occurred in a tightly bounded interval. However, because the laboratory simulation was intentionally restricted to four packets, the observed timing should not be extrapolated as evidence of a sustained SYN-flood rate. Packet rate, duration, persistence, and resulting service impact would need to be established over a substantially larger authorized dataset before concluding that a denial-of-service condition existed.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/bounded_syn_activity.pcapng \
-Y 'tcp.flags.syn==1 && tcp.flags.ack==0' \
-T fields \
-e frame.number \
-e frame.time_epoch \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
| tee reports/syn_timeline.tsv
Warning: program compiled against libxml 215 using older 214
1      1789157062.227471477    127.0.0.1      27322    127.0.0.1      80
4      1789157062.228140336    127.0.0.1      4924     127.0.0.1      80
7      1789157062.228821336    127.0.0.1      18524    127.0.0.1      80
10     1789157062.229357509    127.0.0.1      33781    127.0.0.1      80
```

Quantitative findings so far

Summary of the evidence include:

Indicator	Finding
Initial SYN packets	4
Source IP	127.0.0.1
Destination IP	127.0.0.1
Destination port	TCP/80
Unique source ports	4
SYN-ACK responses	4
Subsequent RST packets	4
Observed SYN span	~1.886 ms
Multiple external/distributed sources	No
Sustained traffic demonstrated	No
Service disruption demonstrated	No

Forensic Interpretation

The captured traffic was examined to determine whether the observed activity was consistent with a SYN flood, a port scan, or a bounded SYN simulation. The evidence identified four initial SYN packets originating from 127.0.0.1 and targeting the authorized Apache service at 127.0.0.1:80. Each SYN received a corresponding SYN-ACK from the server, followed immediately by a TCP RST from the originating client port. Four unique source ports were observed, and all four connection attempts occurred within an approximately 1.886-millisecond interval.

The observed sequence demonstrates deliberate SYN connection activity but does not, by itself, establish a SYN flood or denial-of-service condition. A SYN flood would ordinarily require evidence of sustained or sufficiently high-volume SYN traffic, persistence over time, accumulation of incomplete connections, and/or measurable impact on the availability or performance of the target service. None of these conditions can be established from the four-packet laboratory capture. In addition, all traffic originated from the local loopback address rather than multiple external sources.

The traffic is also not best classified as a port scan because all four SYN packets were directed to the same destination IP address and the same TCP destination port, 80. There was no evidence of sequential probing across multiple ports or hosts.

Based on the available evidence, the most appropriate classification is therefore a bounded SYN-pattern simulation conducted within an authorized laboratory environment. The capture demonstrates the TCP behaviour associated with the controlled SYN activity but does not provide sufficient evidence to conclude that a SYN flood or denial-of-service attack occurred.

Evidence-Based Classification

Classification	Assessment	Evidence
SYN flood / DoS	Not established	Only four SYNs; no sustained duration or service impact demonstrated
Port scan	Not supported	All packets targeted the same host and TCP/80
SYN-pattern activity	Confirmed	Four initial SYNs and four corresponding SYN-ACK responses
Bounded laboratory simulation	Confirmed	Activity was intentionally limited to four authorized packets

Detection and Mitigation Recommendations

Detection Recommendations

Detection of potential SYN-flood activity should focus on identifying abnormal increases in TCP SYN requests relative to completed TCP connections. Network monitoring tools such as Wireshark, TShark, IDS/IPS platforms, firewalls, and network telemetry systems can be used to monitor TCP flags, source addresses, destination ports, connection rates, and incomplete handshakes.

For a potential SYN-flood condition, useful indicators include a sustained increase in SYN packets to a particular service, a high ratio of SYN packets to completed handshakes, a large number of connections remaining in a half-open state, repeated SYN requests from numerous sources, and degradation in the availability or response time of the affected

service. Monitoring should also consider the normal traffic baseline of the environment so that legitimate increases in connection volume are not incorrectly classified as attacks. The TShark filters used in this investigation can support initial packet-level investigation. In particular, the filter: `tcp.flags.syn==1 && tcp.flags.ack==0` can be used to identify initial SYN requests, while `tcp.flags.syn==1 && tcp.flags.ack==1` identifies SYN-ACK responses. Comparing these observations over an appropriate time window can help establish whether a sudden increase in SYN activity is occurring.

Mitigation Recommendations

Where sustained SYN-flood activity is confirmed, mitigation should be applied at the network and host layers according to the organisation's architecture and established incident-response procedures. TCP SYN cookies can be enabled where supported to reduce the amount of server-side state maintained for incomplete TCP handshakes. Firewalls, load balancers, and network security controls may also provide SYN rate limiting or connection-rate controls to restrict abnormal connection attempts.

Where traffic originates from identifiable malicious or unexpected sources, appropriate access-control or filtering mechanisms may be used. For distributed attacks, source-based blocking alone may be ineffective because traffic can originate from many different addresses. In such circumstances, upstream filtering, rate limiting, load balancing, or DDoS protection services may be more appropriate.

Mitigation controls should be carefully tuned against the established normal baseline. Excessively restrictive thresholds may block legitimate users during periods of genuine high demand. Any mitigation action should therefore be accompanied by monitoring of service availability, legitimate connection success rates, and system performance.

Finally, detected events should be correlated with firewall, web-server, operating-system, IDS/IPS, and application logs. Packet captures provide network-level evidence, but determining actual service impact requires corroborating evidence from the affected system.

Evidence and Hash Register

The investigation maintained a structured evidence directory containing the packet captures, extracted analysis results, and case documentation. Evidence files were

retained separately from working and reporting materials to support preservation and reproducibility.

Evidence Register

Evidence / Artifact	Purpose
Lab directory structure	Establishes the organised evidence and working-file structure
Tool verification output	Confirms availability and versions of TShark, Scapy and Apache
Apache TCP/80 verification	Establishes the authorized local target and confirms that Apache was listening on TCP/80
System/environment information	Provides host, date and network-interface context
reports/case_notes.txt	Documents the case identifier, student, course, target and authorized scope
evidence/normal_http.pcapng	Normal HTTP/TCP baseline capture
evidence/bounded_syn_activity.pcapng	Capture of the four-packet controlled SYN activity
reports/syn_ack_responses.tsv	Extracted SYN-ACK response records
reports/ack_reset_candidates.tsv	Extracted ACK/RST candidate records
reports/syn_counts_by_pair.txt	Quantitative count of SYN packets by source/destination pair
reports/unique_syn_source_ports.txt	Unique source ports observed in the SYN activity
reports/syn_timeline.tsv	Timestamp and connection details for the initial SYN packets

CONCLUSION

This investigation successfully examined TCP SYN activity against an authorized local Apache service using TShark in a controlled Kali Linux laboratory environment. A normal

HTTP session was first captured to establish a baseline for expected TCP connection behavior, including the three-way handshake, application traffic, and connection termination.

The subsequent controlled SYN simulation produced four SYN packets directed to 127.0.0.1:80, each originating from a unique source port. All four SYN packets received corresponding SYN-ACK responses from the Apache service and were subsequently followed by TCP RST packets. Quantitative analysis confirmed that the activity was limited to four connection attempts originating from the local host.

Although the four SYN packets occurred within a short time interval, the evidence does not establish a SYN flood or denial-of-service condition. The capture does not demonstrate sustained high-volume traffic, prolonged accumulation of half-open connections, multiple external sources, or measurable degradation of the target service. It also does not support classification as a port scan because all connection attempts targeted the same host and TCP port.

Based on the available evidence, the activity is therefore classified as a bounded SYN-pattern simulation conducted within an authorized laboratory environment. The investigation demonstrates the use of packet filtering, TCP flag analysis, quantitative measurement, baseline comparison, and evidence integrity procedures to support a defensible forensic interpretation of network traffic.